

Autonomous Area Scanning and Precision Landing on a Visual Target with a Parrot Mambo Minidrone

Jatin Satyam

School for Engineering of Matter, Transport and Energy

Ira A. Fulton Schools of Engineering, Arizona State

University, Tempe, AZ

Email: jsatyam@asu.edu

Abstract—This report documents a search-and-land mission: a Parrot Mambo minidrone sweeps an indoor arena in a scanning flight pattern, detects a colored landing pad below using its downward-facing camera, and transitions from search to a centered vertical descent onto the pad. The model derives from the MathWorks Minidrone Competition project structure, with the author's scanning references, color-detection thresholds, and landing handover logic. Detection uses color segmentation and blob centroid extraction on the live camera feed. The deployed flight finds the pad and lands on it autonomously in one continuous take.

Index Terms—Parrot Mambo, Simulink, search and land, color segmentation, blob analysis, precision landing, Minidrone Competition.

I. INTRODUCTION

Line following assumes the path is already beneath the drone. This mission removes that assumption: the target's location is not given, so the drone must first search the arena, then recognize the landing pad, then land on it, chaining three behaviors into one autonomous flight.

The Parrot Mambo carries the sensor suite the whole series relies on: an ultrasound sensor timing pulses to the floor for altitude, a pressure sensor for altitude changes, an IMU providing linear acceleration and angular rates, and a downward-facing camera whose optical flow contributes horizontal velocity estimates. Four motors in an X configuration are commanded through motor mixing: collective thrust, a differential between the clockwise and counter-clockwise pairs for yaw, and side-to-side speed imbalances for pitch and roll. The airframe is underactuated, with six degrees of freedom and only four inputs, so every lateral motion begins as a deliberate tilt of the thrust vector.

II. SYSTEM ARCHITECTURE

All flight code was developed in Simulink against the Parrot Minidrone Support Package and deployed to the physical airframe, keeping the model structure the support package expects: an estimator consuming the sensor buses, a flight control system, and reference logic that the author modified per task. Structuring the work as small reusable subsystems made iteration and debugging on hardware considerably less painful.

The model is built on the Minidrone Competition project structure, which supplies the airframe, estimator, and flight control scaffolding; the author's contribution lives in the scanning flight references, the vision thresholds tuned to the pad's signature color, and the logic that arbitrates between search, centering, and descent.

III. METHOD

A. Five-mode mission state machine

The mission logic is a five-mode state machine: TAKEOFF_STABILIZE climbs to hover height ignoring the camera; SCAN flies a lawnmower pattern across the arena; HOVER_ALIGN centers the drone over the detection; DESCEND lowers altitude with the lateral position frozen; and DONE holds position after touchdown. Transitions are guarded by stability and lost-target counters so a single noisy frame cannot flip the mission state.

B. Robust detection

Blue detection combines two methods and accepts a pixel if either fires: channel dominance, requiring blue to exceed red and green by margins while rejecting dark and white surfaces, and calibrated thresholds from the Color Thresholder as backup. The centroid extractor is codegen-safe and applies hysteresis to the found flag, with separate switch-on and switch-off area thresholds to prevent flickering detections.

C. Centering and landing

The centroid offset from the image center drives lateral corrections in HOVER_ALIGN; once stable over the pad, the logic freezes X and Y and descends onto the target.

IV. FLIGHT TESTING AND RESULTS

In the deployed flight over the checkerboard arena, the drone scans, acquires the colored pad, centers itself, and lands on the target autonomously, completing the full search-detect-land chain in a single continuous flight.



Fig. 1. The search-and-land flight: approaching the colored pad on the arena.

V. CHALLENGES

- Detecting a small pad from a moving, vibrating camera without false positives from the arena's own pattern.
- Transitioning cleanly from scanning motion to a centered descent without overflying the target.
- Color thresholds holding up across the arena's lighting.

VI. CONCLUSION

The mission demonstrates behavior arbitration on top of the series' vision and control stack: search, recognition, and precision landing chained autonomously on real hardware.

APPENDIX: MATLAB FUNCTION CODE

The following MATLAB Function blocks are the author's custom code from the deployed Simulink model, reproduced verbatim.

Listing 1. detectBlue: dual-method robust color detection

```
function mask = detectBlue(R, G, B)
%detectBlue Robust blue detection combining two methods
% Works across different cameras and lighting conditions.
% Method 1: Blue channel dominance (B must be strongest channel by margin)
% Method 2: Calibrated thresholds (backup, from Color Threshold)
% A pixel is "blue" if EITHER method detects it.

% ===== METHOD 1: Blue Dominance (most reliable) =====
% Blue must be the strongest channel AND reasonably bright
% Using uint16 to avoid uint8 overflow/underflow
Ru = uint16(R);
Gu = uint16(G);
Bu = uint16(B);

% Blue must be at least 40 higher than red
blueOverRed = (Bu > Ru + uint16(40));

% Blue must be at least 20 higher than green
blueOverGreen = (Bu > Gu + uint16(20));

% Blue channel must be at least 60 (not too dark)
blueBright = (Bu > uint16(60));

% Red must be below 180 (reject white/bright surfaces)
redLow = (Ru < uint16(180));

dominance = blueOverRed & blueOverGreen & blueBright & redLow;

% ===== METHOD 2: Calibrated Thresholds (backup) =====
% From Color Thresholder app – fixed to require high blue
calibrated = (R <= uint8(100)) & ...
(G <= uint8(160)) & ...
(B >= uint8(80));

% ===== COMBINE: pixel is blue if either method says so =====
mask = dominance | calibrated;

end
```

Listing 2. CentroidFromMask: codegen-safe centroid with hysteresis

```
function [found, cx, cy, area] = CentroidFromMask(mask)
% Codegen-safe centroid + area + stable found (hysteresis)
% mask: binary image (120x160), uint8/logical

% ----- Tunable thresholds -----
AREA_ON = uint32(400); % was 800 % found turns ON when area >
= AREA_ON % was 800, now calibrated for your sheet at 0.7
m
AREA_OFF = uint32(250); % was 500 % found turns OFF when area
<= AREA_OFF % was 500, hysteresis gap prevents flickering
g
% -----

% Ensure mask is treated as 120x160 (codegen expects fixed size)
% (If your image size is different, change the loop limits + center below.)
area = uint32(0);
sx = uint32(0);
sy = uint32(0);

for r = 1:120 % rows (Y)
    for c = 1:160 % cols (X)
        if mask(r,c) ~= 0
            area = area + 1;
            sx = sx + uint32(c);
            sy = sy + uint32(r);
        end
    end
end

% Centroid (if no pixels, return image center)
if area > 0
    cx = single(double(sx) / double(area));
    cy = single(double(sy) / double(area));
else
    cx = single(80); % center x for width 160
    cy = single(60); % center y for height 120
end

% ----- Stable found using hysteresis -----
persistent foundState
if isempty(foundState)
    foundState = false;
end
```

```

if ~foundState
    if area >= AREA_ON
        foundState = true;
    end
else
    if area <= AREA_OFF
        foundState = false;
    end
end

found = foundState;
% -----
end

Listing 3. ScanCenterDescend: five-mode mission state machine
function [xRef,yRef,zRef,done] = ScanCenterDescend(found,cx,cy,
    area,xEst,yEst,zEst)
% =====
% STATE MACHINE: Takeoff -> Scan -> Align -> Stabilize -> Desce
nd -> Done
% =====
% Mode 0: TAKEOFF_STABILIZE - climb to hover height, ignore cam
era
% Mode 1: SCAN - lawn mower pattern searching for bl
ue
% Mode 2: HOVER_ALIGN - center over blue centroid
% Mode 3: DESCEND - lower altitude with X/Y frozen
% Mode 4: DONE - hold position, flight complete
% =====
% ----- enforce scalar types -----
xEst = single(xEst(1));
yEst = single(yEst(1));
zEst = single(zEst(1));
cx = single(cx(1));
cy = single(cy(1));
area = uint32(area(1));
found = uint8(found(1));

% outputs (defaults, will be overwritten below)
xRef = single(0);
yRef = single(0);
zRef = single(0);
done = uint8(0);

% ===== PERSISTENT STATE =====
persistent mode x0 y0 xR yR zR yDir stableCount lostCount xHold
yHold tickCount

if isempty(mode)
    mode = uint8(0);
    x0 = xEst;
    y0 = yEst;
    xR = xEst;
    yR = yEst;
    zR = single(-0.7);
    yDir = int8(1);
    stableCount = uint16(0);
    lostCount = uint16(0);
    xHold = xEst;
    yHold = yEst;
    tickCount = uint32(0);
end

% ===== TUNABLE PARAMETERS =====

% --- Takeoff ---
TAKEOFF_TICKS = uint32(600);
MIN_ALT = single(-0.55);

% --- Blue detection ---
AREA_DETECT = uint32(250);
AREA_LOCK = uint32(350);

% --- Camera center and tolerance ---
CX0 = single(80);
CY0 = single(60);
CX_TOL = single(10);
CY_TOL = single(10);

% --- Scan pattern ---
YSPAN = single(1.8);
XSPAN = single(1.8);
yStep = single(0.0009);
xLaneStep = single(0.25);

% --- Stability ---
STABLE_N = uint16(12);

```

```

    LOST_N = uint16(20);

% --- Alignment control ---
K_CX = single(0.0008);
K_CY = single(0.0008);
MAX_XY_STEP = single(0.008);

% --- Heights ---
zHover = single(-0.7);
zStep = single(0.0005);
zTarget = single(-0.50);
zDoneThr = single(-0.53);

% ===== HELPER FLAGS =====
atAltitude = (zEst <= MIN_ALT);
blueSeen = (found ~= 0) && (area >= AREA_DETECT) && atAltitud
e;
centeredOk = (found ~= 0) && (area >= AREA_LOCK) && atAltitude
&& (abs(cx - CX0) <= CX_TOL) && (abs(cy - CY0) <= CY_TOL);

% ===== STATE MACHINE =====
if mode == uint8(0)
% ===== MODE 0: TAKEOFF & STABILIZE =====
    xR = x0;
    yR = y0;
    zR = zHover;

    tickCount = tickCount + uint32(1);

    if (tickCount >= TAKEOFF_TICKS) && atAltitude
        mode = uint8(1);
        x0 = xEst;
        y0 = yEst;
        xR = xEst;
        yR = yEst;
    end

elseif mode == uint8(1)
% ===== MODE 1: SCAN (lawnmower pattern) =====
    zR = zHover;

    yR = yR + single(yDir) * yStep;

    if yR >= (y0 + YSPAN)
        yR = y0 + YSPAN;
        yDir = int8(-1);
        xR = xR + xLaneStep;
    elseif yR <= y0
        yR = y0;
        yDir = int8(1);
        xR = xR + xLaneStep;
    end

    if xR >= (x0 + XSPAN)
        xR = x0;
    end

    if blueSeen
        mode = uint8(2);
        stableCount = uint16(0);
        lostCount = uint16(0);
        xR = xEst;
        yR = yEst;
        zR = zHover;
    end

elseif mode == uint8(2)
% ===== MODE 2: HOVER & ALIGN TO CENTROID =====
    zR = zHover;

    if blueSeen
        lostCount = uint16(0);

        ex = cx - CX0;
        ey = cy - CY0;

        dY = -K_CX * ex;
        dX = -K_CY * ey;

        if dX > MAX_XY_STEP
            dX = MAX_XY_STEP;
        end
        if dX < -MAX_XY_STEP
            dX = -MAX_XY_STEP;
        end
        if dY > MAX_XY_STEP
            dY = MAX_XY_STEP;
        end
        if dY < -MAX_XY_STEP

```

```

        dY = -MAX_XY_STEP;
    end

    xR = xEst + dX;
    yR = yEst + dY;

    if centeredOk
        stableCount = stableCount + uint16(1);
    else
        stableCount = uint16(0);
    end

    if stableCount >= STABLE_N
        mode = uint8(3);
        xHold = xEst;
        yHold = yEst;
        xR = xHold;
        yR = yHold;
        zR = zHover;
    end
else
    lostCount = lostCount + uint16(1);
    stableCount = uint16(0);

    if lostCount >= LOST_N
        mode = uint8(1);
    end
end

elseif mode == uint8(3)
    % ===== MODE 3: DESCEND =====
    xR = xHold;
    yR = yHold;

    zR = zR + zStep;
    if zR > zTarget
        zR = zTarget;
    end

    if zEst >= zDoneThr
        mode = uint8(4);
        done = uint8(1);
    end
end

else
    % ===== MODE 4: DONE =====
    xR = xHold;
    yR = yHold;
    zR = zTarget;
    done = uint8(1);
end

% ===== OUTPUT =====
xRef = xR;
yRef = yR;
zRef = zR;
end

```

ACKNOWLEDGMENT

The author completed this work as part of the RAS 546 curriculum in the MS Robotics and Autonomous Systems program at Arizona State University, and thanks the course staff for guidance on the Parrot Mambo platform, flight-arena logistics, and safe flight operations.

REFERENCES

- [1] MathWorks, “Simulink Support Package for Parrot Minidrones,” MathWorks documentation, 2025.
- [2] MathWorks, “Getting Started with Simulink for Parrot Minidrone Hover,” MathWorks examples, 2025.
- [3] B. Douglas, “Drone Simulation and Control” video series, MathWorks / Control Systems in Practice.